TrueForge Coding Agent: 4-Minute Demo Video Walkthrough Script

Turn-by-Turn Production Script & On-Screen Verification Guide | Techfest 2026-27, IIT Bombay & TrueFoundry Forge-a-thon

Target Codebase: pallets/flask @ 3.1.3 (commit 22d92470)	**Duration Limit:** Strictly < 4:00 minutes
GitHub: github.com/Pranav060307/TrueFoundry-Forge-a-thon	**Harness:** @truefoundry/trueforge (port 8790)

OFFICIAL COMPETITION VIDEO RULES (PDF Pages 5–6):
- **Duration:** Exactly under 4 minutes (max 04:00). Videos exceeding 4:00 risk disqualification.
- **Task Sequence:** Tasks **A**, **B**, and **C** must be shown in that **exact order**.
- **Verification:** Every task must be followed immediately by its on-screen terminal verification command.
- **TrueForge Inspection:** The TrueForge session inspector (turns and tool calls) must be briefly shown in Task B.
- **No Cut Policy:** You may fast-forward model generation time, but do NOT cut between prompt submission and diff appearance.

■■ Master Timeline & Scene Summary

Scene	Timestamp	Core Task & Actions	On-Screen Verification
Scene 1	00:00 – 00:14	System health check, status bar handshakes	TrueForge :8790 & Sidecar :4545 online
Scene 2	00:15 – 00:52	**Task A:** Inline Edit on get_debug_flag	FLASK_DEBUG=off returns False
Scene 3	00:53 – 02:08	**Task B:** Agent Mode unprompted across 3+ files	git diff --stat (3 files), flask config masked JSON, pytest 56 passed
Scene 4	02:09 – 02:35	Token streaming & interactive cancellation	Instant Stop, git status clean
Scene 5	02:36 – 03:25	**Task C:** Resilience & Transparent Auto Fallback	401 auth caught, transparent switch to secondary
Scene 6	03:26 – 04:00	Full test suite run & repository transparency	npm test 5/5 passed, Write-up PDF shown

Scene 1: Introduction & System Health Handshake (00:00 – 00:14 (Duration: 14s))

Visual Focus:	VS Code editor open with pallets/flask workspace. Status bar and Agent Panel visible.
Key Actions:	• Point mouse to status bar showing **TrueForge: Connected (:8790)** and **Sidecar: Healthy (:4545)**. • Briefly reveal the TrueForge Agent panel in the left sidebar showing green status badges.
Voiceover:	*"This is TrueForge Coding Agent, an editor-native coding assistant built strictly on TrueFoundry's TrueForge harness. All reasoning runs on TrueForge at localhost:8790, while editor capabilities are exposed as MCP tools via a local Python sidecar."*

Scene 2: Task A – Inline Edit & Diff Review (src/flask/helpers.py) (00:15 – 00:52 (Duration: 37s))

Visual Focus:	Open src/flask/helpers.py, navigate to line 26 (def get_debug_flag()).
Key Actions:	• Select get_debug_flag and press **Ctrl+I** (or click ■ **Task A** in webview header). • Submit prompt: *'Treat the values "off" and "n" as false as well, case-insensitively, and update the docstring to list every value that is treated as false.'* • Virtual diff editor (trueforge-diff://) opens side-by-side with 0-line context Myers hunks. • Demonstrate per-hunk review: click **[✓ Accept Hunk]** (or use title bar check icon).
Voiceover:	*"For Task A, we trigger an inline edit with Ctrl+I. TrueForge stages surgical Myers diff hunks strictly in memory. We inspect the proposed changes side-by-side and accept them hunk-by-hunk."*

Terminal:	`$ FLASK_DEBUG=off python -c "from flask.helpers import get_debug_flag; print(get_debug_flag())"`
	`  ■ Output: False (Baseline returned True)`
	`$ FLASK_DEBUG=n python -c "from flask.helpers import get_debug_flag; print(get_debug_flag())"`
	`  ■ Output: False`
	`$ FLASK_DEBUG=1 python -c "from flask.helpers import get_debug_flag; print(get_debug_flag())"`
	`  ■ Output: True`

Scene 3: Task B – Autonomous Agent Mode Across 3+ Files (flask config) (00:53 – 02:08 (Duration: 75s))

Visual Focus:	Switch focus to sidebar Agent Panel. Terminal ready below for verification.
Key Actions:	• Submit unprompted goal (no files named) or click ■ **Task B (3 Files)**: *'Add a new built-in CLI command flask config that prints the loaded application configuration as JSON, masking the values of any key containing SECRET, PASSWORD, TOKEN or KEY as "***". It must appear in flask --help, be covered by a test, and be documented alongside the other built-in commands.'* • Show **Context Engine Table**: BM25 ranker + AST outlines within 4,000-token budget. • Agent autonomously invokes MCP tools (search_workspace, read_file, propose_write). • Briefly switch to browser/terminal showing the **TrueForge Session** at localhost:8790. • Review virtual diffs touching 3 files (cli.py, test_cli.py, docs/cli.rst) and click **[Accept All Hunks]**.
Voiceover:	*"In Task B, the agent operates autonomously with no files named. It queries our MCP tools, retrieves context under a 4,000 token budget, and stages edits across three files. Notice the TrueForge session inspector tracking all turns. Once accepted, we run tests."*

Terminal:	`$ git diff --stat`
	■ *Output: 3 files changed, 60 insertions(+) (docs/cli.rst, src/flask/cli.py, tests/test_cli.py)*
	`$ flask --help`
	■ *Output: Commands list includes: config (Show current application configuration)*
	`$ flask config`
	■ *Output: Outputs formatted JSON with SECRET_KEY, APP_SECRET, API_KEY masked as "***"*
	`$ pytest tests/test_cli.py -q`
	■ *Output: 56 passed, 3 skipped in 0.54s (All CLI tests green)*

Scene 4: Streaming Tokens & Instant Safe Cancellation (02:09 – 02:35 (Duration: 26s))

Visual Focus:	Agent Panel in sidebar. Terminal open alongside.
Key Actions:	• Type long exploratory prompt: *'Analyze all routes, blueprints, and request handlers across the entire Flask codebase and summarize them in detail.'* • Observe SSE tokens streaming in real time. • Click the **Stop (Cancel)** button mid-generation. • Show generation immediately terminates; VS Code remains completely responsive. • Run git status in terminal to verify zero dirty writes touch the disk.
Voiceover:	*"Tokens stream live over Server-Sent Events. Clicking Stop instantly halts the turn via TrueForge's session cancellation endpoint. The editor never blocks, and disk state remains 100% clean."*
Terminal:	`$ git status` ■ *Output: nothing to commit, working tree clean (Zero silent or corrupted writes)*

Scene 5: Task C – Resilience & Transparent Auto Fallback (02:36 – 03:25 (Duration: 49s))

Visual Focus:	Settings / Agent Panel showing model failover in action.
Key Actions:	• Point primary model to broken endpoint (or click ■ **Task C (Failover)**). • Repeat Task A instruction on src/flask/helpers.py. • Observe: Primary model returns 401 Unauthorized / Unreachable Provider. • Extension's ModelFailoverManager transparently intercepts failure. • Non-blocking notification appears in status bar: *'Transparently switched to fallback model'*. • Fallback model seamlessly completes the edit without user intervention or editor locks.
Voiceover:	*"Task C proves resilience. When our primary model encounters a broken endpoint or 401 error, our client intercepts it and transparently routes the turn to the backup model without throwing blocking modals or corrupting session state."*
Terminal:	`$ git status` ■ *Output: helpers.py modified (Only valid accepted diffs present, zero corrupted state)*

Scene 6: Conclusion & Repository Transparency (03:26 – 04:00 (Duration: 34s))

Visual Focus:	Terminal running test suite, then quick display of GitHub repository and Write-up PDF.
Key Actions:	• Execute full test suite in terminal: npm test (all 5 suites pass). • Execute backend test suite: python tests/test_backend.py (10/10 pass). • Briefly display the 3-page Write-up PDF (docs/TrueForge_Coding_Agent_Writeup.pdf). • Show public GitHub repository with MIT License and clean commit log.
Voiceover:	*"Every milestone is verified with automated test suites. Built strictly from scratch for Techfest 2026-27 IIT Bombay. Zero silent writes, model-agnostic, and 100% powered by TrueForge."*

Terminal:	`$ npm test`
	■ *Output: 5/5 test suites passed (Client, Myers Diff, Task A, Task B, Task C)*
	`$ python tests/test_backend.py`
	■ *Output: 10/10 MCP tool tests passed successfully*

■ Practical Recording & Submission Tips

Resolution & FPS	Record at 1080p (1920x1080) at 30 or 60 fps. Set VS Code font size to 14–16px for crisp readability.
Terminal Pre-fill	Pre-run terminal verification commands so you can recall them instantly using Up-Arrow + Enter.
1-Click Demo Triggers	Use the webview header buttons (■ **Task A**, ■ **Task B**, ■ **Task C**) to trigger verified demo states flawlessly without typing latency.
Submission Format	Upload the video as **Unlisted** on YouTube or open-permission Google Drive link before **20th Nov 2026, 23:59 IST**.